

Aplikacija za proceduralno generiranje višinskih zemljevidov in digitalnih modelov terena

Strokovno poročilo

Mentor: David Zindović, prof.

Avtor: Nik Pavlović-Semen R 4. D

Ljubljana, april 2026

Abstract

This report presents the development of an application for terrain generation that can then be used in other digital media. The initial part of the report presents the basic structure of the application and the process of visualizing terrain created by the user. This is then followed by a description of the system that allows the user to assemble the terrain generation process, as well as its composition and operation. The subsequent section presents the methods and techniques used in terrain generation, such as erosion simulation, and their implementation in the application. The final part describes the systems for exporting terrain data and saving projects created within the application.

Keywords: procedural generation, terrain, height map, erosion simulation, Godot, C#

Povzetek

V tem poročilu je predstavljen razvoj aplikacije za generacijo terena, ki se lahko nato uporabi v drugih digitalnih medijih. V začetnem delu poročila je predstavljena osnovna zgradba aplikacije in postopek vizualizacije terena, ki ga ustvari uporabnik. Temu sledi opis sistema, ki uporabniku omogoča sestavljanje postopka generacije terena, ter njegova sestava in delovanje. Podrobneje so predstavljene tudi povezave med posameznimi deli sistema. V osrednjem delu so predstavljene metode in tehnike, uporabljene pri ustvarjanju terena, kot je simulacija erozije, ter njihova izvedba v aplikaciji. V zadnjem delu so opisani še sistemi za izvoz terena in shranjevanje projektov, ustvarjenih v aplikaciji.

Ključne besede: proceduralna generacija, teren, višinski zemljevid, erozijska simulacija, Godot, C#

Kazalo vsebine

Kazalo vsebine	3
1. Uvod	4
2. Izbira orodij in programskih jezikov	4
3. O aplikaciji	5
4. Urejevalnik in vizualizacija	6
4. 1. Konstrukcija modela	6
4. 2. Izvedba konstrukcije modela	7
4. 2. Dodatki vizualizaciji	8
4. 2. 1. Kamera	8
4. 2. 2. Senčilnik za določanje materiala terena	9
4. 3. Urejevalnik	10
5. Node graph sistem	11
5. 1. Vozlišče (node)	11
5. 2. Povezovanje	12
6. Inšpektor vozlišč	16
7. Funkcionalnost vozlišč	17
8. Generacija terena in vrste funkcionalnosti	19
8. 1. Osnovna elevacija	19
8. 2. Redistribucija	21
8. 3. Distorzija	22
8. 4. Terasa	23
8. 5. Simulacija erozije	23
8. 6. Ostale funkcionalnosti	24
9. Izvoz	25
10. Shranjevanje	26
11. Zaključek	27
12. Viri in literatura	28
IZJAVA O AVTORSTVU	29

1. Uvod

V razvoju 3D vsebin in iger je pogosto potreben teren. Ta pogosto nastopa kot območje oziroma virtualni svet, v katerem se odvijajo dogodki (kar velja predvsem za videoigre), ali pa samo kot del ozadja. Na spletu obstaja nekaj aplikacij, ki so namenjene profesionalnemu ustvarjanju terena in te uporabljajo nekaj zanimivih načinov generiranja le tega. Nedavno sem se za te načine začel zanimati in jih raziskovati, zato sem se odločil narediti to aplikacijo, ki je predstavljena v tem poročilu. Namenjena je proceduralni generaciji raznovrstnega terena, ki se lahko izvozi in nadaljnje uporabi v razne namene. Uporabnik lahko ustvarja teren z uporabo t. i. *node graph* sistema, aplikacija pa potem naredi še vizualizacijo ustvarjenega.

2. Izbira orodij in programskih jezikov

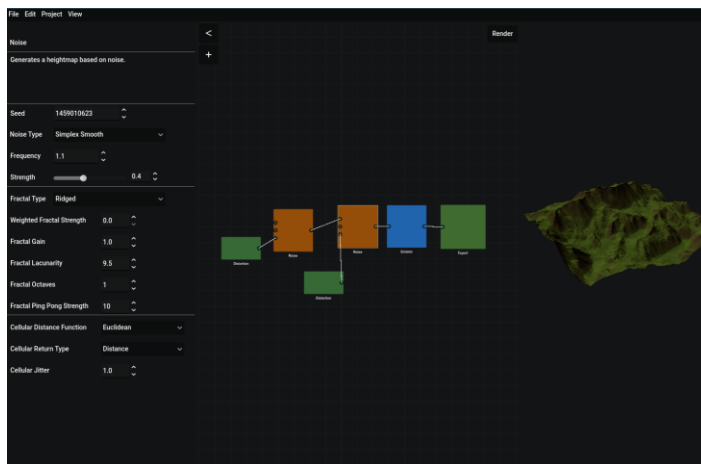
Aplikacija je narejena v odprtokodnem igralnem pogonu Godot, ki je izdan pod licenco MIT. Ker poleg razvoja videoiger omogoča tudi razvoj aplikacij in imam z njim že nekaj izkušenj, je za to aplikacijo idealna izbira. Dovoljuje ustvarjanje uporabniških vmesnikov in 3D-upodabljanje, kar aplikacija uporablja za prikaz generiranega digitalnega modela reliefa. Aplikacija izkorišča tudi nekaj modulov oziroma knjižnic (predvsem FastNoiseLite za uporabo matematičnega šuma), ki so že vključene v Godot.

Aplikacija je sprogramirana v dveh jezikih, C# in GDScript. Za generacijo višinskih zemljevidov, terena, izvajanje večjih operacij nad njim (kot je npr. simulacija erozije) in izvoz se uporablja C#, medtem ko se za uporabniške vmesnike, urejevalnik terena, konstrukcijo in prikaz modelov ter drugo, kar se ne navezuje na samo generacijo, uporablja jezik GDScript.

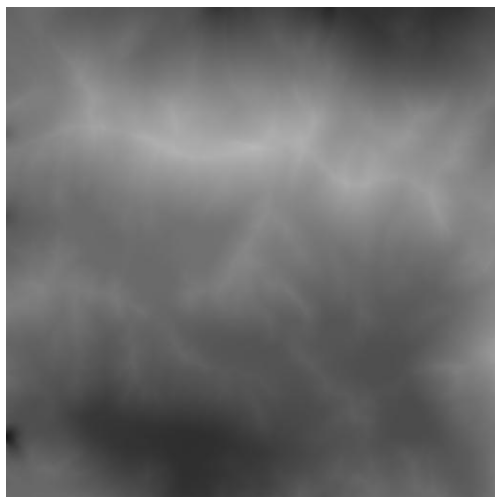
3. O aplikaciji

Uporabniku je v aplikaciji omogočeno ustvarjanje terena v urejevalniku, v katerem uporabnik postavlja in povezuje med seboj vozlišča (node), objekte, ki nad vrednostmi terena izvedejo določene operacije. Vsak node ima drugačno funkcijo, od generacije z matematičnim šumom do simulacije erozije ali pa izvoza terena. Uporabnik lahko izbere katerikoli node in aplikacija na tisti točki v procesu izvajanja operacij nad terenom naredi vizualizacijo terena kot 3D elevacijski model.

Aplikacija teren obravnava kot mrežo vrednosti, v kateri ima vsaka celica svojo višinsko vrednost. Če govorimo o tem konceptu kot črno-bela slika je to višinski zemljevid (angl. heightmap), katerih izvoz aplikacija tudi omogoča, saj so najbolj pogost način za uvažanje in digitalno shranjevanje terena.



Slika 1 Zaslonska slika aplikacije



Slika 2 Podoba višinskega zemljevida

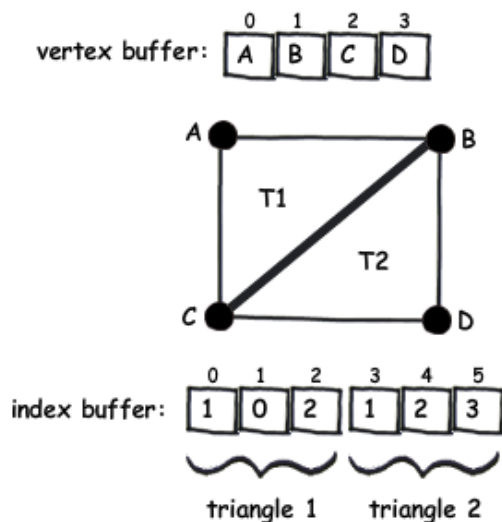
4. Urejevalnik in vizualizacija

Aplikacija je razdeljena na urejevalnik in na okno za 3D vizualizacijo. V tem poglavju je podrobneje opisano delovanje obeh. Ker je to del uporabniškega vmesnika Godot vsebuje nekaj uporabnih razredov, kot so vidna polja oziroma pogledi (viewport). Urejevalnik in vizualizacijsko okno sta oba svoja objekta ali scene, ta sta v aplikaciji naenkrat v enem oknu prikazana s pomočjo razreda *viewport*.

4. 1. Konstrukcija modela

3D modeli so običajno sestavljeni iz trikotnikov, ki jih računalnik prikaže kot celoten model. Ti so sestavljeni iz oglišč in povezav med njimi (index). Omenjena oglišča oziroma stičišča (ang. vertex, v množini vertices) so točke v prostoru, kjer se srečata dve ali več črt oziroma robov. Povezave med njimi (ang. index, v množini indices) so trikotniki sestavljeni iz skupin treh oglišč, saj te ustvarijo trikotno ploskev.

Da programsko ustvarimo model terena, je potrebno definirati oglišča glede na koordinate trenutne točke v terenu. Ker je teren definiran kot mreža vrednosti lahko vsako polje v mreži deluje kot oglišče, in njegova vrednost kot višina tega oglišča. Te povežemo med sabo v skupine po tri. Več trikotnikov, ki skupaj tvorijo sklenjeno telo ali ploskev ustvarijo 3D model.



Slika 3 Primer sestavljanja modela

4. 2. Izvedba konstrukcije modela

V oknu za vizualizacijo je primerek objekta 3D modela. Za konstrukcijo modela terena ta vsebuje funkcijo `generate_terrain`, ki sprejme dvodimenzionalno tabelo, ki vsebuje vrednosti od 0 do 1 in predstavlja generiran teren. Za urejanje modela sta uporabljena pomožna objekta, ki jih vsebuje Godot.

```
var a_mesh: ArrayMesh
var surfaceTool = SurfaceTool.new()
```

Slika 4 Objekta, uporabljena pri urejanju modela

Funkcija iterira čez teren in ustvari oglišča na kordinatah, ki se ujemajo z pozicijo točke v tabeli terena in njeno vrednostjo, pomnoženo z maksimalno višino nastavljeno v nastavitvah aplikacije.

```
surfaceTool.begin(Mesh.PRIMITIVE_TRIANGLES)
for z in range(z_size+1):
    for x in range(x_size+1):
        var y = heightmap[z][x] * max_height

        surfaceTool.set_uv(Vector2(inverse_lerp(0, x_size, x), inverse_lerp(0, z_size, z)))
        surfaceTool.add_vertex(Vector3(x,y,z))

        #if visualization:
            #draw_sphere(Vector3(x,y,z))

for row in range(z_size):
    for col in range(x_size):
        var upper_left = row * (x_size + 1) + col #row + etc. bcs indexes are a one dimensional array
        var upper_right = row * (x_size + 1) + col + 1
        var lower_left = (row + 1) * (x_size + 1) + col
        var lower_right = (row + 1) * (x_size + 1) + col + 1

        surfaceTool.add_index(upper_left)
        surfaceTool.add_index(upper_right)
        surfaceTool.add_index(lower_left)
        surfaceTool.add_index(upper_right)
        surfaceTool.add_index(lower_right)
        surfaceTool.add_index(lower_left)

surfaceTool.generate_normals()
a_mesh = surfaceTool.commit()
mesh = a_mesh
```

Slika 5 Ustvarjanje oglišč in povezav med njimi

Da se pri odprtju aplikacije prikaže ravna ploskev, ker teren še ni bil generiran, funkcija sama generira tabelo terena z ničelnimi vrednostmi in jo prikaže.

4. 2. Dodatki vizualizaciji

Model terena pa ni prikazan samo statično. Možno si ga je podrobno ogledati preko objekta kamere in ima tudi teksturo, da ni samo ene barve.

4. 2. 1. Kamera

V oknu za vizualizacijo se lahko uporabnik obrača okoli modela terena in mu približa pogled. To je narejeno preko kamere. Ker so omejitve kamere drugačne glede na velikost modela, se bojo spremenljivke, ki določajo omejitve kamere, hitrost približevanja in podobno spreminjale. Za to skrbi funkcija *setCameraProperties*, ki se kliče vsakič ko hočemo na novo prikazati teren. S tem se ponastavita tudi rotacija in pozicija kamere.

Rotacija in pozicija kamere delujeta tako, da aplikacija zazna pritisk na miški in v primeru povečave vrtenje kolesčka na miški. Ko zazna pritisk na miški vsako osvežitev nastavi rotacije kamere glede na trenutno rotacijo, premik miške med pritiskom in občutljivostjo (ta je lastnost objekta). Rotacija kamere na eni izmed osi je omejena.

```
@export_group("View Settings")
@export var cameraRotateSensitivity = 0.01
@export var cameraZoomSensitivity = 1
@export var zoom_speed = 20
@export var zoom_increment = 250

var camera_target_y: float = 1500
var camera_max_y = 2000
var camera_min_y = 10

func _ready():
    Globals.terrain_view = self
    setCameraProperties()

func setCameraProperties():
    cameraPivot.position.x = Globals.terrain_size.x/2
    cameraPivot.position.z = Globals.terrain_size.y/2

    camera.position.y = Globals.terrain_size.x * 2
    camera_target_y = camera.position.y
    camera_max_y = camera.position.y * 1.25 + 250

    cameraPivot.rotation.x = 45
    cameraPivot.rotation.y = 45
```

Slika 6 Lastnosti kamere

```
func _unhandled_input(event):
    if event is InputEventMouseMotion:
        if event.button_mask == MOUSE_BUTTON_MASK_LEFT || event.button_mask == MOUSE_BUTTON_MASK_MIDDLE:
            cameraPivot.rotation.y = cameraPivot.rotation.y - event.relative.x * cameraRotateSensitivity
            cameraPivot.rotation.x = clamp(cameraPivot.rotation.x - event.relative.y * cameraRotateSensitivity, 0, PI/2.5)

func _input(event):
    if event is InputEventMouseButton:
        if event.is_pressed():
            if event.button_index == MOUSE_BUTTON_WHEEL_UP:
                if camera.position.y > camera_min_y:
                    camera_target_y = clamp(camera.position.y-zoom_increment, camera_min_y, camera_max_y)
                    set_physics_process(true)
            elif event.button_index == MOUSE_BUTTON_WHEEL_DOWN:
                if camera.position.y < camera_max_y:
                    camera_target_y = clamp(camera.position.y+zoom_increment, camera_min_y, camera_max_y)
                    set_physics_process(true)

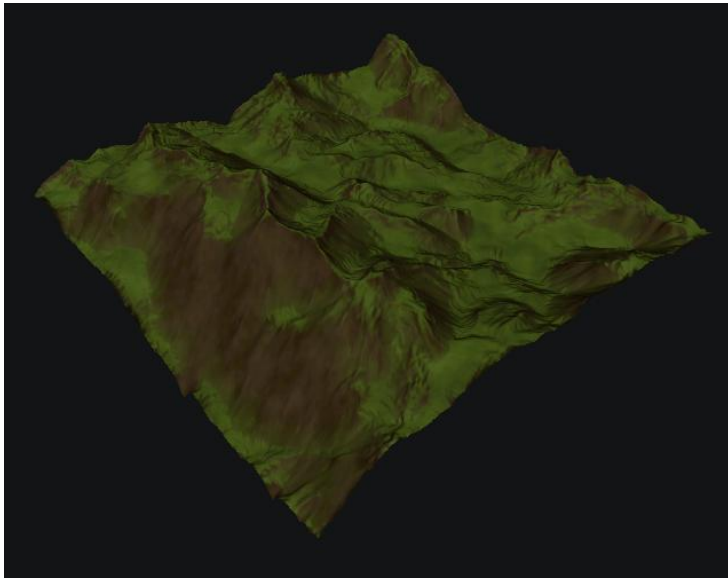
func _physics_process(delta):
    if is_equal_approx(camera.position.y, camera_target_y):
        camera.position.y = camera_target_y
        set_physics_process(false)
    else:
        camera.position.y = lerp(camera.position.y, camera_target_y, zoom_speed*delta)
```

Slika 7 Odsek kode, ki skrbi za rotacijo in povečavo pri kameri

Ko se zazna vrtenje kolesčka na miški in bližina pogleda ne bi šla čez ali pod omejitve se nastavi ciljna bližina, ki se ji z interpolacijo preko uporabe funkcije *lerp* približuje bližina pogleda, dokler nista obe bližini enaki.

4. 2. 2. Senčilnik za določanje materiala terena

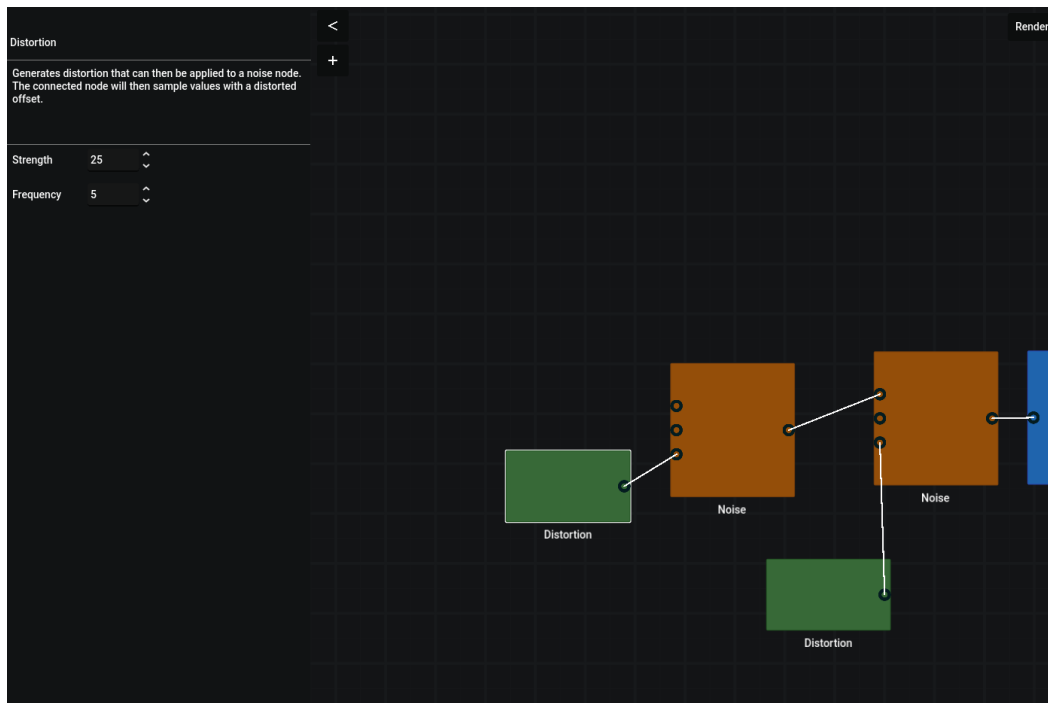
Teren ni povsod enake barve. Za to poskrbi fragmentacijski senčilnik (fragment shader), ki izračuna barvo piksla pred prikazom. Ta izvede kalkulacije nad modelom da izve naklon terena in tam uporabi primeren material (se opazi kot obarvan matematični šum). Maskiranje naklona se izvaja z izračunom skalarne produkta normal mreže modela terena in transformirane matrike pogleda.



Slika 8 Slika terena, pri katerem so strmejši deli obarvani drugače

4. 3. Urejevalnik

V urejevalnem oknu, ki se nahaja levo od vizualizacijskega, lahko uporabnik ureja *node graph* sistem, da ureja teren. Uporabnik se lahko premika po delovnem prostoru urejevalnika. Pri tem se dejansko premika kamera, podobno kot pri vizualizacijskem oknu. Korenski objekt urejevalnika je v projektu zelo pomemben, saj veže skupaj večino aplikacije. Vsebuje funkcije, ki upravljajo z *node graph* sistemom, nastavitvami, podokni urejevalnika in drugo.



Slika 9 Urejevalnik

Hierarhično je urejevalnik sestavljen iz korenskega objekta, ki nadzoruje vse sisteme, ki ne generirajo terena, dveh objektov, ki s tem pomagata, ozadja (je samo ponavljajoča se slika), kontajnerjev za druge uporabniške vmesnike (na levi) in kamere.

5. Node graph sistem

Vse v zvezi z vozlišči nadzira »Node graph« sistem. Ta upravlja s povezovanjem in celotnim življenjskim ciklom vozlišča.

5. 1. Vozlišče (node)

V aplikaciji je vozlišče (ali *node*) objekt v urejevalniku, ki ga lahko povežemo z drugimi. Vsako vozlišče generira svojo mrežo vrednosti ali pa nad tisto, ki jo dobi na svoj vhod, izvede določene operacije ali simulacije. Ko skupaj povežemo več vozlišč dobimo graf, pri katerem je zadnje najbolj desno vozlišče konec grafa, kjer se izvede zadnja operacija nad terenom. S povezovanjem in nastavljanjem vozlišč uporabnik ustvarja teren.

Vsaka vrsta vozlišča je svoj razred, ki je izpeljan iz baznega razreda *EditorNode*, ki sem ga napisal kot podlago za vozlišča. Vsak izpeljani razred vsebuje enega ali več izhodov in vhodov, instanco razreda, ki določa funkcionalnost vozlišča, in nekaj grafičnih komponent. Ko se v urejevalniku naredi nova instanca vozlišča se ta čez skripto urejevalnika registrira. V tabelo, ki vsebuje identifikatorje vozlišč se shrani identifikator novega vozlišča, da ga lahko skripte dosežejo pozneje.

```
func register_node(node: EditorNode, id = null):
>|  if id == null:
>|    node.id = global_node_id
>|    #nodes[node.id] = node
>|    nodes.set(node.id, node)
>|    global_node_id += 1
>|  else:
>|    node.id = id
>|    nodes.set(node.id, node)
>|    if global_node_id <= id:
>|      global_node_id = id + 1
>|
>|  print(node.id)
```

Slika 10 Odsek registracije vozlišč

Ko se vozlišče registrira se inicializira tudi nekaj signalov vozlišča, ki so uporabljeni za obveščanje drugih skript o dogajanju z instanco vozlišča.

```

func initing():
> | if not inited:
> |     print(id)
> |     move_node_to_top.connect(Globals.editor.move_node_to_top.bind())
> |     select_node.connect(Globals.editor.select_node.bind())
> |     functionality.parentID = id;
> |     inited = true

```

Slika 11 Inicializacija po registraciji

Vozlišča se da tudi izbrati s klikom na njih. Ko je vozlišče izbrano, ta obvesti korensko skripto urejevalnika, ki posodobi svoj zapis trenutnega izbranega vozlišča, ga premakne na vrh vrstnega reda prikazovanja in prikaže pravilen inšpektor (več o tem v poglavju o inšpektorju). Po izbiri aktivnega vozlišča se prikaže tudi grafični element, ki nakazuje izbrano vozlišče.

```

func select():
> | selected = true
> | node_highlight.visible = true

func unselect():
> | functionality.Deselect()
> | selected = false
> | node_highlight.visible = false

func _on_gui_input(event):
> | if event is InputEventMouseButton:
> |     if event.pressed:
> |         drag_position = get_global_mouse_position() - global_position
> |         emit_signal("move_node_to_top", self)
> |         if not selected:
> |             emit_signal("select_node", self)
> |     else:
> |         drag_position = null
> | if event is InputEventMouseMotion and drag_position:
> |     global_position = get_global_mouse_position() - drag_position
> |     emit_signal("moved_node")

```

Slika 12 Izbira aktivnega vozlišča na strani instance vozlišča

5. 2. Povezovanje

Vsako vozlišče vsebuje vsaj en izhod ali vhod. Te predstavljajo konektorji, objekti, ki so uporabljeni pri povezovanju vozlišč. Med sabo se lahko povežejo samo konektorji nasprotni vrste (vhod - izhod) in enake kategorije. Kategorija označuje vrsto spremenljivk v vsakem polju izhodne tabele na izhodu npr. decimalna vrednost ali dvodimenzionalni vektor. Vsak konektor ima zato dve spremenljivki naštetega tipa (enum), ki določata vrsto in kategorijo konektorja. Vozlišča shranjujejo vse svoje konektorje v tabeli, in ko potrebujejo referenco na konektor čez njo iterirajo in preverjajo identifikator konektorja.

Vozlišča imajo funkcije, ki vrnejo svoj identifikator ali referenco do konektorja glede na dobljeni identifikator konektorja. Čez konektorje je možno pridobiti tudi identifikator konektorja samega ali vozlišča, kateremu pripada.

Ko uporabnik klikne na konektor in povleče z miško urejevalnik prejme signal, da naj začne povezovati. V primeru da povezava na konektorju že obstaja se s pomočjo sistema pridobivanja vozlišč in konektorjev iz njihovih identifikatorjev obstoječa povezava izbriše. Povezave so svoj razred, ki shranjuje povezani vozlišči in njune povezane konektorje.

```

> func start_connecting(from: NodeConnector):
  connecting = true
  connecting_from = from
  connecting_to = null
  for c_connection in connections:
    if (c_connection.from_connector == from.get_id() and c_connection.from_node == from.get_node_id()) or (c_connection.to_connector == from.get_id() and c_connection.to_node == from.get_node_id()):
      remove_connection(connections.find(c_connection))

```

Slika 13 Začetek povezovanja na strani skripte urejevalnika

Ko uporabnik spusti gumb na miški se povezovanje preneha. Če je miška takrat bila nad drugim konektorjem, se preveri veljavnost povezave in se ta ustvari.

```

func create_connection(from: NodeConnector, to: NodeConnector):
  if is_cycle(from.parent.get_id(), to.parent.get_id()):
    return
  elif is_cycle(to.parent.get_id(), from.parent.get_id()):
    return
  if from.IOCategory != to.IOCategory:
    return
  for c_connection in connections:
    if (c_connection.to_connector == to.get_id() and c_connection.to_node == to.get_node_id()) or (c_connection.from_connector == to.get_id() and c_connection.from_node == to.get_node_id()):
      remove_connection(connections.find(c_connection))
  var connection: Connection = Connection.new()
  if from.IOType == Globals.NodeConnectorIOType.OUTPUT and to.IOType == Globals.NodeConnectorIOType.INPUT: #FROM is not output, TO is not input
    connection.from_node = from.get_node_id()
    connection.to_node = to.get_node_id()
    connection.from_connector = from.get_id()
    connection.to_connector = to.get_id()
  else:
    connection.from_node = to.get_node_id()
    connection.to_node = from.get_node_id()
    connection.from_connector = to.get_id()
    connection.to_connector = from.get_id()
  connections.append(connection)
  from.connect_connectors(to)
  to.connect_connectors(from)

```

Slika 14 Ustvarjanje povezave

Pred ustvarjanjem povezave med vozliščema se preveri ujemanje kategorij in vrst konektorjev. Preveri se tudi, da se ni ustvarila povezava, ki bi ustvarila neskončno zanko. To se zgodi, ko povežemo dve vozlišči tako, da se izhodni konektor poveže na vozlišče, ki se v grafu nahaja pred vozliščem iz katerega povezujemo. Skripta ob vsakem poskusu povezovanja rekurzivno prečka celoten graf in preverja, če do vozlišča, ki ga povezujemo, že obstaja pot. To naredi v obe smeri, naprej in nazaj v grafu.

```

#region Cycle detection
##Returns if a path already exists between two nodes.
func is_cycle(from_node: int, to_node: int):
    return path_exists_forward(from_node, to_node, {}) or path_exists_backwards(from_node, to_node, {})
## Recursive function that checks path existence. CALL is_cycle INSTEAD.
func path_exists_forward(from_node: int, to_node: int, visited: Dictionary) -> bool:
    if from_node == to_node:
        return true
    visited[to_node] = true
    for c in connections:
        if c.from_node == to_node:
            var next_id = c.to_node
            if not visited.has(next_id):
                if path_exists_forward(from_node, next_id, visited):
                    return true
    return false
## Recursive function that checks path existence. CALL is_cycle INSTEAD.
func path_exists_backwards(from_node: int, to_node: int, visited: Dictionary) -> bool:
    if from_node == to_node:
        return true
    visited[from_node] = true
    for c in connections:
        if c.to_node == from_node:
            var next_id = c.from_node
            if not visited.has(next_id):
                if path_exists_backwards(next_id, to_node, visited):
                    return true
    return false

```

Slika 15 Rekurzivno preverjanje validnosti povezave

Če se povezava uveljavi kot pravilna se ustvari nov objekt v katerega se shranijo identifikatorji vozlišč in povezanih konektorjev. Za vsako povezavo se ustvari tudi objekt razreda, ki med dvema konektorjema riše črto. Ko se črta ustvari pridobi reference na ciljne konektorje, da lahko od njih pridobiva informacije o njihovih pozicijah in spremembah. Črte nadzira upravljalni objekt pod urejevalnikom, ki sledi dodajanju, brisanju in posodabljanju črt. Ob premikanju vozlišč se posodabljaajo tudi začetne in končne pozicije črt, enako se črte brišejo ko se zbriše povezava med vozliščema ali vozlišče samo.

```

var from_node_extracted: EditorNode = Globals.editor.nodes.get(from_node)
from_connector_extracted = from_node_extracted.get_connector_by_id(from_connector)

var to_node_extracted: EditorNode = Globals.editor.nodes.get(to_node)
to_connector_extracted = to_node_extracted.get_connector_by_id(to_connector)

begin_cap_mode = Line2D.LINE_CAP_ROUND
end_cap_mode = Line2D.LINE_CAP_ROUND
joint_mode = Line2D.LINE_JOINT_ROUND

width = 2
default_color = Color(1.0, 1.0, 1.0, 1.0)
z_index = 5

from_offset = from_connector_extracted.center_offset * from_connector_extracted.get_parent().scale
to_offset = to_connector_extracted.center_offset * to_connector_extracted.get_parent().scale

add_point(from_connector_extracted.global_position+from_offset)
add_point(to_connector_extracted.global_position+to_offset)

to_node_extracted.connect("moved_node", update_positions)
from_node_extracted.connect("moved_node", update_positions)
get_viewport().size_changed.connect(update_positions)

Globals.editor.connection_line_manager.remove_all.connect(remove)

```

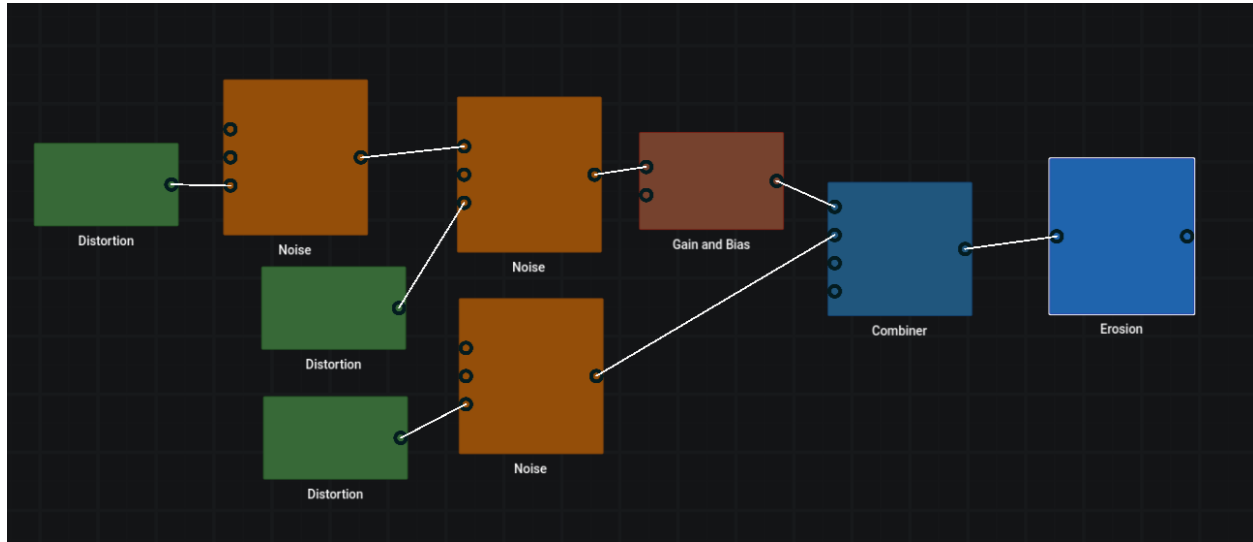
Slika 16 Proces, ki ga izvede vsaka črta, ko se ustvari

```

func update_positions():
>| if from_connector_extracted == null or to_connector_extracted == null:
>| >| queue_free()
>| >| return
>| points[0] = from_connector_extracted.global_position+from_offset
>| points[1] = to_connector_extracted.global_position+to_offset

```

Slika 17 Posodabljanje krajišč črt na vsaki instanci črte



Slika 18 Graf vozlišč

Izbrano vozlišče lahko uporabnik tudi izbriše. Proces brisanja vozlišča pregleda vse povezave, vezane na to vozlišče, jih zbere in nato vse izbrane povezave izbriše. Šele nato se izbriše vozlišče.

```

if selected_node:
>| var connections_to_delete: Array[Connection]
>| for connection: Connection in connections:
>| >| if connection.from_node == selected_node.id or connection.to_node == selected_node.id:
>| >| >| connections_to_delete.append(connection)
>| for connection in connections_to_delete:
>| >| remove_connection(connections.find(connection))
>|
>| nodes.erase(nodes.find_key(selected_node))
>| selected_node.unselect()
>| node_inspector.queue_free()
>| selected_node.queue_free()

```

Slika 19 Brisanje vozlišča

6. Inšpektor vozlišč

Da lahko uporabnik spreminja lastnosti vozlišč in dodatno prilagaja teren, aplikacija vsebuje inšpektorje. Vsako vozlišče ima svoj inšpektor, ki je uporabniški vmesnik. Ta prikazuje in dovoli spremembo nastavitev vozlišča. Vsak inšpektor je izpeljan iz baznega razreda, ki vsebuje kodo za upravljanje z vnosnimi polji.

Ko uporabnik izbere vozlišče skripta urejevalnika uporabi pot, shranjeno v vozlišču, da odpre inšpektor in vsako vnosno polje se posodobi z informacijami, ki jih inšpektor pridobi iz objekta funkcionalnosti vozlišča preko metod za pridobivanje in nastavljanje lastnosti.

```
if (node_inspector):
>| node_inspector.queue_free()
var inspector_scene = load(selected_node.inspector)
node_inspector = inspector_scene.instantiate()
inspector_container.add_child(node_inspector)
```

Slika 20 Ena izmed funkcionalnosti za nastavljanje trenutnega inšpektorja

```
for child in inputs:
>| if child is InspectorInput:
>| >| child.connect("updated", field_updated)
>| >| child.set_value(Globals.editor.selected_node.get_property(child.property))
```

Slika 21 Posodabljanje vnosnih polj

Pred prvo izbiro vozlišča je v nosilcu za uporabniški vmesnik inšpektorja postavljen začasni inšpektor, ki ni povezan z vozliščem.

Vnosna polja so objekti, ki ob spremembi vrednosti o tem obvestijo inšpektor. Ta potem naprej obvesti vozlišče, kjer se nastavi spremenjena lastnost. Vsaka vrsta vnosnega polja ima metode za pridobivanje in nastavljanje svoje vrednosti ter ime lastnosti, ki jo bo sprememba polja posodobila na strani vozlišča.

7. Funkcionalnost vozlišč

Funkcionalnosti so razredi, katerih instance so podrejene vozliščem. Vsaka vrsta vozlišča ima drugačno vrsto funkcionalnosti, ki je izpeljana iz skupnega baznega razreda. Ti objekti omogočajo generacijo in izvajanje operacij nad terenom.

Funkcionalnosti vsebujejo slovar oziroma asociativno tabelo lastnosti, ki jih uporabljajo pri operacijah. Vozlišča lahko z uporabo funkcij komunicirajo s svojimi funkcionalnostmi in spreminjajo njihove lastnosti. Te se uporabljajo za spreminjanje nastavitev funkcionalnosti iz inšpektorja, za serializacijo in shranjevanje vozlišč. Bazni razred funkcionalnosti vsebuje tudi druge metode, ki se izvedejo ob drugih dogodkih (npr. inicializaciji).

```
func set_property(property, value):
> | functionality.properties[property] = value

func get_property(property):
> | if functionality.properties.has(property):
> | > | return functionality.properties[property]
> | return null

func get_properties():
> | return functionality.properties
```

Slika 22 Par funkcij za komunikacijo lastnosti funkcionalnosti z vozliščem

Ko uporabnik izbere vozlišče in zažene generacijo, funkcionalnost izbranega vozlišča zažene kodo, ki preveri vsak vhod in s pomočjo urejevalnikovega sistema povezav in identifikatorjev pokliče evaluacijske funkcije funkcionalnosti vozlišč, ki so povezane na izbrano vozlišče.

```
public T[,] getFromInput<T>(int to_port) {
    GodotObject editor = (GodotObject)GetNode<Node>("/root/Globals").Get("editor");
    GodotObject connection = (GodotObject)editor.Call("get_connection_to", parentID, to_port);
    if (connection == null) {
        T[,] empty = {};
        return empty;
    }
    NodeFunctionality otherFunctionality = ((NodeFunctionality)((GodotObject)editor.Call("get_node_connected_to", parentID, to_port)).Get("functionality"));
    return otherFunctionality.Evaluate<T>((int)connection.Get("connector_from"));
}
```

Slika 23 Pridobivanje podatkov iz vhoda

Ker morajo funkcionalnosti vozlišč, napisane v jeziku C# dostopati do funkcij, ki so napisane v drugem jeziku (GDScript), se tukaj uporablja medjezično kodiranje oziroma funkcije, namenjene za komunikacijo med skriptami, ki so napisane v teh dveh jezikih.

Kodo za pridobivanje podatkov iz vhodov potem izvede vsaka funkcionalnost vozlišč v grafu in podatke obdela. Tako deluje tok podatkov v grafu. Vozlišča prosijo druga vozlišča, povezana na vhode, za podatke, in jih nato obdelajo.

Podatki se obdelujejo z evaluacijsko funkcijo, virtualno funkcijo, ki jo vsaka izpeljana funkcionalnost prepiše. V tej funkciji se kliče prej omenjena koda za pridobivanje podatkov iz vhodov, saj evaluacijska funkcija *Evaluate* te pridobljene podatke obdela.

```
public virtual T[,] Evaluate<T>(int port){
    float[,] a = {};
    a = getFromInput<float>(0);
    if (a.GetLength(0) == 0) {

        Vector2 terrain_size = (Vector2)GetNode<Node>("/root/Globals").Get("terrain_size");
        float[,] flat = new float[(int)terrain_size.X, (int)terrain_size.Y];
        for (int i = 0; i < flat.GetLength(0); i++) {
            for (int j = 0; j < flat.GetLength(1); j++) {
                flat[i, j] = (float)properties["foo"];
            }
        }
        a = flat;
    }
    else {
        for (int i = 0; i < a.GetLength(0); i++) {
            for (int j = 0; j < a.GetLength(1); j++) {
                a[i, j] += (float)properties["foo"];
            }
        }
    }
    return (T[,])(object)a;
}
```

Slika 24 Osnovna evaluacijska funkcija

Teren se med funkcionalnostmi prenaša kot C# tabela, ampak skripta za vizualizacijo terena kot model sprejema drugačno podatkovno vrsto (posebna GDScript tabela). Ob pridobivanju podatkov iz funkcionalnosti za vizualizacijo se zato kliče koda za pretvarjanje podatkov v primerno obliko za vizualizacijo.

```
public static Godot.Collections.Array ConvertToGDArray(float[,] arrayToConvert) {
    Godot.Collections.Array convertedArray = [];
    for (int i = 0; i < arrayToConvert.GetLength(0); i++) {
        Godot.Collections.Array innerArray = [];
        for (int j = 0; j < arrayToConvert.GetLength(1); j++) {
            innerArray.Add(arrayToConvert[i, j]);
        }
        convertedArray.Add(innerArray);
    }
    return convertedArray;
}
```

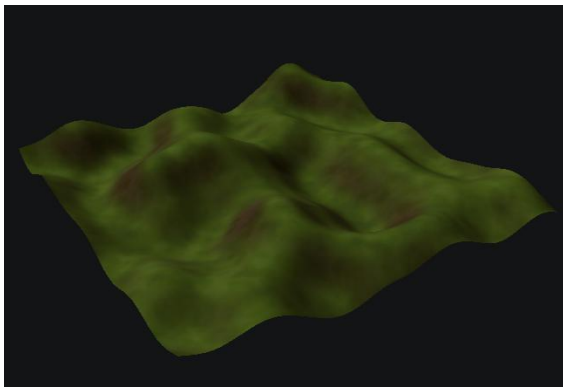
Slika 25 Konverzija oblike zapisa terena

8. Generacija terena in vrste funkcionalnoti

Poleg izjem vsaka funkcionalnost upodablja svoj koncept, ki se uporablja pri generaciji terena. Nekatere funkcionalnosti vozlišč so namenjene zgolj enostavnejšim opravičom, na primer združevanju več vhodnih vrednosti v en izhod.

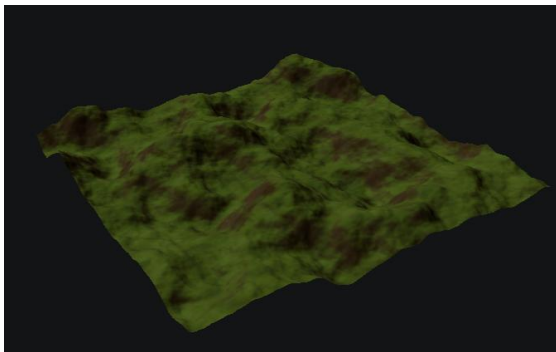
8. 1. Osnovna elevacija

Pogost način generiranja osnovne elevacije terena je z matematičnim šumom. Uporabljen šum izhaja iz matematičnih funkcij. Za razliko od normalnega šuma, ki vsaki točki dodeli nepovezano naključno število, ta šum deluje kot gradient, zato se vrednosti postopoma spreminjajo, in so med seboj povezane.



Slika 26 Navaden Perlinov šum upodobljen kot 3D model

Ena plast navadnega šuma ne izgleda resnično in ne vsebuje veliko podrobnosti, zato se v praksi skupaj uporabi več plasti šumov različnih frekvenc, te imenujemo oktave.



Slika 27 Več zmnoženih plasti Perlinovega šuma različnih frekvenc

Ta koncept ima v aplikaciji svoje vozlišče in funkcionalnost kot »*Noise node*«. Ko se kliče evaluacijska funkcija te funkcionalnosti, koda iz vnosov pridobi teren. Če na vhodu nima povezav prvo ustvari teren kjer je vrednost vsakega polja 0. Potem vsakemu polju prišteje vrednost šuma, ki se nahaja na enaki koordinati, kot polje v tabeli. Generacija šuma je narejena s *FastNoiseLite* objektom oziroma knjižnico.

Lastnosti funkcionalnosti tega vozlišča vsebujejo številne nastavitve šuma in njegovih oktav.

```
ThreadData tdata = (ThreadData)data;
FastNoiseLite tnoise = new FastNoiseLite();
tnoise.Seed = (int)properties["Seed"];
tnoise.CellularDistanceFunction = (FastNoiseLite.CellularDistanceFunctionEnum)(int)properties["CellularDistanceFunction"];
tnoise.CellularJitter = (float)properties["CellularJitter"];
tnoise.CellularReturnnType = (FastNoiseLite.CellularReturnnTypeEnum)(int)properties["CellularReturnnType"];
/*tnoise.DomainWarpAmplitude = (float)properties["DomainWarpAmplitude"];
tnoise.DomainWarpEnabled = (bool)properties["DomainWarpEnabled"];
tnoise.DomainWarpFractalGain = (float)properties["DomainWarpFractalGain"];
tnoise.DomainWarpFractalLacunarity = (float)properties["DomainWarpFractalLacunarity"];
tnoise.DomainWarpFractalOctaves = (int)properties["DomainWarpFractalOctaves"];
tnoise.DomainWarpFrequency = (float)properties["DomainWarpFrequency"];
tnoise.DomainWarpFractalType = (FastNoiseLite.DomainWarpFractalTypeEnum)(FastNoiseLite.FractalTypeEnum)(int)properties["DomainWarpFractalType"];
tnoise.DomainWarpType = (FastNoiseLite.DomainWarpTypeEnum)(int)properties["DomainWarpType"];*/
tnoise.FractalGain = (float)properties["FractalGain"];
tnoise.FractalLacunarity = (float)properties["FractalLacunarity"];
tnoise.FractalOctaves = (int)properties["FractalOctaves"];
tnoise.FractalPingPongStrength = (float)properties["FractalPingPongStrength"];
tnoise.FractalType = (FastNoiseLite.FractalTypeEnum)(int)properties["FractalType"];
tnoise.Frequency = (float)properties["Frequency"];
tnoise.NoiseType = (FastNoiseLite.NoiseTypeEnum)(int)properties["NoiseType"];
tnoise.Offset = (Vector3)properties["Offset"];

tnoise.DomainWarpEnabled = false;

for (int y = tdata.StartY; y < tdata.EndY; y++)
{
    for (int x = 0; x < PrimaryMap.GetLength(0); x++)
    {
        PrimaryMap[x, y] = GetValue(x, y, tnoise, tdata.UseDistortion, tdata.UseMask, tdata.CreatePrimaryMap);
    }
}
```

Slika 28 Odlomek funkcionalnosti šuma na niti procesorja

Kot poskus pohitritve koda generacije osnovne elevacije deluje na večih procesorskih nitih. Za vsako uporabljeno nit se ustvari struktura, ki vsebuje podatke, ki so potrebni za delovanje kode na večih nitih.

```
private struct ThreadData
{
    2 references
    public int StartY;
    2 references
    public int EndY;
    2 references
    public bool CreatePrimaryMap;
    2 references
    public bool UseDistortion;
    2 references
    public bool UseMask;
}
```

Slika 29 Omenjena struktura

Vsaka nit dobi za obdelati del mreže terena in za vsako polje se kliče funkcija, ki vzorči šum in hkrati uporabi masko ali distorzijo. Skoraj vsaka funkcionalnost vsebuje uporabo mask, ki določajo kako močno se na vsaki poziciji pozna učinkovitost operacije. Na kratko, distorzija deluje kot zamik vzorčenja, ki je drugačen za vsako polje.

```
float strength = (float)properties["Strength"];
float current = 0f;
if (!CreatePrimaryMap)
{
    current = PrimaryMap[x, y];
}

if (UseDistortion)
{
    Vector2 distortion = DistortionMap[x, y];
    current += (tnoise.GetNoise2D(x + distortion.X, y + distortion.Y)+ 1) / 2 * strength;
}
else
{
    current += (tnoise.GetNoise2D(x, y)+ 1) / 2 * strength;
}

if (UseMask)
{
    current *= MaskMap[x, y];
}

return Math.Clamp(current, 0f, 1f);
```

Slika 30 Odlomek vzorčenja šuma

Samo šum deluje dobro kot osnova za teren, vendar neobdelan ne izgleda najboljše ali prepričljivo.

8. 2. Redistribucija

Za ustvarjanje ravnih dolin se redistribucijsko vozlišče višinske vrednosti potencira, tako nastanejo višje vrednosti terena postanejo bolj strme. Višje vrednosti eksponenta povzročajo premik srednjih višin proti nižjim legam, medtem ko nižje vrednosti povzročajo premik proti višjim legam. Redistribucijsko vozlišče hkrati tudi množi vrednosti terena in s tem spreminja izrazitost višin.

8. 3. Distorzija

Zaradi načina nastanka resničnega terena ima ta pogosto deformiran in malo zviti izgled. Za doseg podobnega učinka je uporabljena distorzija. Običajno se višina celice pridobi na naslednji način, kjer je h višina in x in y sta pozicija polja:

$$h = \text{noise}(x, y)$$

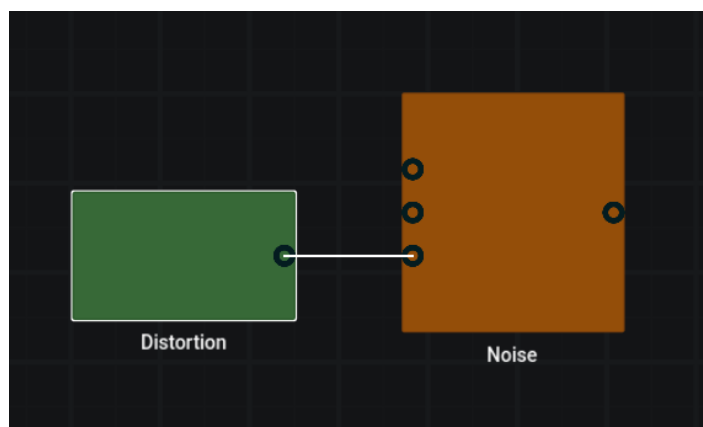
Funkcionalnost vozlišča distorzije je, da generira tabelo, ki vsebuje dvodimenzionalne vektorje, ki predstavljajo odmike. Ustvarita se dva matematična šuma in za odmike na prvi vodoravni osi se vzorči en, za odmike na drugi osi pa drug. Vozlišče, ki ustvarja osnovno elevacijo prejme tabelo odmikov in višino izračuna po enačbi:

$$h = \text{noise}(x + \text{offset}_x, y + \text{offset}_y)$$

Enačbo bi se dalo napisati tudi kot:

$$h = \text{noise}(x + \text{noise}_1(x_1, y_1), y + \text{noise}_2(x_2, y_2))$$

Tako se uporabljata dva enodimenzionalna matematična šuma za emulacijo dvodimenzionalnega, ki bi ga drugače potrebovali za ustvariti odmik za premik točke v dveh dimenzijah.



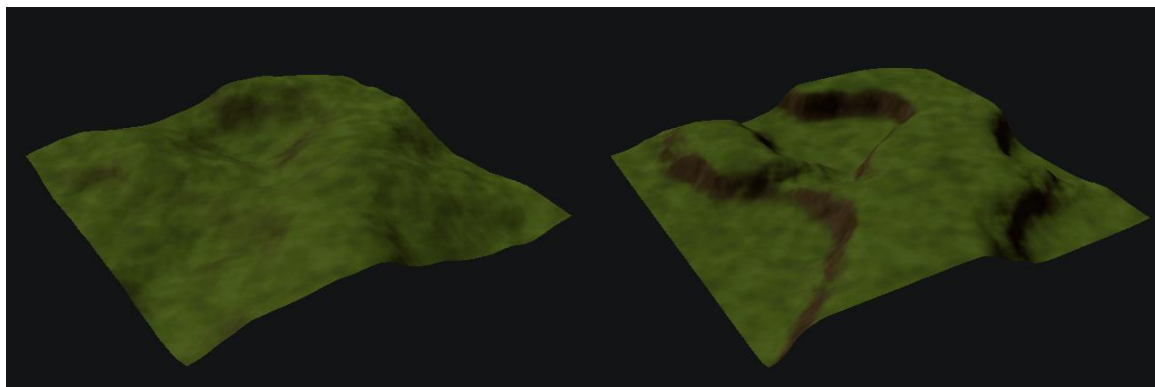
Slika 31 Vozlišči za distorzijo in šum

8. 4. Terasa

Ker ima naravni teren pogosto strme klife in terase, je v aplikaciji implementirano tudi vozlišče za njihovo generacijo. Evaluacijska funkcija funkcionalnosti za ustvarjanje teras vsako vrednost ustavi v naslednjo enačbo, kjer je h nova višina, h_p prejšnja višina, $round$ zaokroževalna funkcija, s strmost robov teras in t lastnost, ki vpliva na število teras:

$$h = \frac{\left(round(h_p) + 0.5 * \left(2 * (h_p - round(h_p))^{2 * s - 1} \right) \right)}{t}$$

Večina drugih enačb za ustvarjanje teras uporabljajo vsaj eno kotno funkcijo. Ker ta enačba ne uporablja kotnih funkcij deluje hitreje. To je pomembno, saj se izvede velikokrat. Če je uporabljena maska, se za izvede tudi funkcija *lerp*, ki interpolira med prejšnjo in novo višino. Ta proces omogoča, da se terase ne morajo ustvarjati povsod, vendar le na določenih pozicijah.



Slika 32 Osnovna elevacija iz šuma

Slika 33 Elevacija iz slike 32 obdelana s terasnim vozliščem

8. 5. Simulacija erozije

V naravi je erozija izjemno pomemben dejavnik pri oblikovanju terena. Najbolj opazna oblika erozije je verjetno hidravlična erozija, ki nastane zaradi tekoče vode, ki oblikuje teren. Ko dežne kapljice padejo na teren in po njem tečejo ga obrabljajo in znižujejo. Med tem ko kapljice tečejo navzdol nabirajo v sebi delce, ki jih kličemo sediment. Kjer teren ni več strm, voda teče počasneje in sediment v kapljicah se odloži, kar poviša teren v tisti točki.

Hidravlično erozijo se lahko simulira na več načinov. V tej aplikaciji je uporabljen način, ki simulira vsak delec vode posamezno. Ta način je relativno počasen in za doseg dobrih rezultatov je treba simulirati več tisoč kapljic, vendar za namene te aplikacije je zadovoljiv.

Funkcionalnost vozlišča za erozijo večkrat naključno na teren postavi delec vode. Kapljica se giblje glede na izračunan gradient spremembe višine. Ko se premika navzdol nižja teren in povečuje količino sedimenta, ki ga vsebuje. Če preseže svojo kapaciteto sedimenta, se ga nekaj odloži nazaj na teren.

Da kapljica lahko nižja teren vse okoli sebe in ne samo na točki kjer se nahaja se uporablja »brush«, ki ima shranjene vse celice terena, na katere lahko iz trenutne pozicije kapljice erozija vpliva. Odlaganje sedimenta tega sistema ne uporablja, nižanje terena pa.

```
if (sediment > sedimentCapacity || deltaHeight > 0) {  
  
    float amountToDeposit = (deltaHeight > 0) ? Mathf.Min (deltaHeight, sediment) : (sediment - sedimentCapacity) * depositSpeed;  
    sediment -= amountToDeposit;  
  
    map[nodeX, nodeY] += amountToDeposit * (1 - cellOffsetX) * (1 - cellOffsetY);  
    map[nodeX + 1, nodeY] += amountToDeposit * cellOffsetX * (1 - cellOffsetY);  
    map[nodeX, nodeY + 1] += amountToDeposit * (1 - cellOffsetX) * cellOffsetY;  
    map[nodeX + 1, nodeY + 1] += amountToDeposit * cellOffsetX * cellOffsetY;  
  
} else {  
    float amountToErode = Mathf.Min ((sedimentCapacity - sediment) * erodeSpeed, -deltaHeight);  
  
    // Use erosion brush to erode from all nodes inside the droplet's erosion radius  
    for (int brushPointIndex = 0; brushPointIndex < erosionBrushIndices[dropletIndex].Length; brushPointIndex++) {  
        Vector2I nodeIndex = erosionBrushIndices[dropletIndex][brushPointIndex];  
        float weighedErodeAmount = amountToErode * erosionBrushWeights[dropletIndex][brushPointIndex];  
  
        float deltaSediment = (map[nodeIndex.X, nodeIndex.Y] < weighedErodeAmount) ? map[nodeIndex.X, nodeIndex.Y] : weighedErodeAmount;  
        map[nodeIndex.X, nodeIndex.Y] -= deltaSediment;  
        sediment += deltaSediment;  
    }  
}
```

Slika 34 Erozija in odlaganje sedimenta

8. 6. Ostale funkcionalnosti

Aplikacija vsebuje tudi številna druga vozlišča s svojimi funkcionalnostmi. Te so:

- Funkcionalnost za združevanje, ki sešteje vse vnose in je namenjeno združevanju posameznih plasti terena.
- Funkcionalnost, ki na več izhodov posreduje vrednost enega izhoda. Ta je uporabna v primeru, ko hočemo izhod vozlišča uporabiti za več stvari, npr. teren se uporabi tudi kot maska.

- Funkcionalnost za normalizacijo, ki normalizira teren. Prejšnja najvišja vrednost postane najvišja možna vrednost in prejšnja najnižja vrednost postane najnižja možna vrednost. Ta funkcionalnost je uporabna, ko želimo, da teren pokriva vse vrednosti od 0 do 1.
- Funkcionalnost za naknadno uporabo maske
- Funkcionalnost za ustvarjanje gradientne maske, ki ustvari masko glede na uporabnikovo izbiro oblike. Ta funkcionalnost je uporabna, ko želimo da se robovi terena bližajo vrednosti 0. Nizki robovi pomagajo pri integraciji terena v medije.
- Funkcionalnost za izvažanje, ki omogoča izvažanje terena

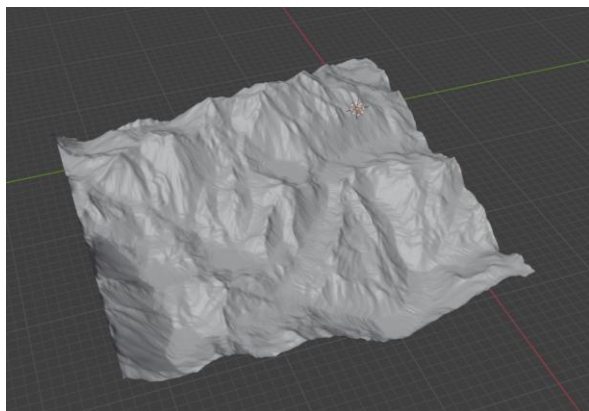
9. Izvoz

Za uporabo generiranega terena v drugih medijih ga je potrebno iz aplikacije izvoziti. To omogoča vozlišče za izvoz. Ker je njegova naloga izvoz končnega rezultata nima izhodov in se uporablja na koncu, ko so vse predhodne operacije že bile izvedene nad terenom. Teren je mogoče izvoziti kot sliko v obliki višinskega zemljevida ali kot 3D model v formatu .glb. Ko se teren izvaža kot višinski zemljevid, funkcionalnost vozlišča za izvoz po generiranju terena ustvari sliko, kjer vsakemu pikslu posebej nastavi vrednost glede na višino ustrezne celice terena.

```
for (int x = 0; x < heightmap.GetLength(0); x++)
{
    for (int y = 0; y < heightmap.GetLength(1); y++)
    {
        float value = heightmap[x, y];
        image.SetPixel(x, y, new Color(value, value, value));
    }
}
```

Slika 35 Odlomek generacije slike višinskega zemljevida

Če teren shranjujemo kot model dejanski izvoz poteka na skripti vizualizacije terena. Ta uporabi funkcijo *generate_export_terrain*, ki je podobna funkciji za konstrukcijo modela, saj obe naredita model. Funkcija za izvoz pa dodatno še izvozi model terena s pomočjo razredov za upravljanje z GLTF datotekami.



Slika 36 Model terena, uvožen v program Blender

10. Shranjevanje

Trenutni projekt lahko uporabnik tudi shrani v datoteko. V datoteki so shranjeni podatki o vseh vozliščih, njihovih povezavah in nastavitvah. Shranijo se tudi nastavitve projekta, kot je na primer velikost terena. Projekt se kasneje lahko uvozi in naložijo se nastavitve in vsa vozlišča, kot so bila nastavljena ob shranjevanju.

Shranjevanje in uvažanje nadzira urejevalniku podrejen objekt *NodeLoadingManager*. Ob shranjevanju skripta objekta *NodeLoadingManager* serializira vsa vozlišča, povezave in nastavitve v JSON format. Pridobljeni JSON potem zapiše v datoteko.

Da aplikacija razlikuje vozlišča, povezave in nastavitve ima vsak zapis v datoteki tip, ki nakazuje na vrsto shranjene vsebine v JSON objektu.

```
for cur_node in Globals.editor.nodes:
    M var snode: SerializedNode = SerializedNode.new()
    M var cnode: EditorNode = Globals.editor.get_node_from_id(cur_node)
    M if !cnode:
    M     print("Error saving project")
    M     return
    M snode.filepath = cnode.filepath
    M snode.node_position = cnode.position
    M snode.properties = cnode.get_properties()
    M snode.id = cur_node
    M var node_data = snode.call("save")
    M var json_string = JSON.stringify(node_data)
    M save_file.store_line(json_string)
for connection: Connection in Globals.editor.connections:
    M var connection_data = {"type": "connection", "from_node": connection.from_node, "to_node": connection.to_node, "from_connector":
    M     connection.from_connector, "to_connector": connection.to_connector}
    M save_file.store_line(JSON.stringify(connection_data))
```

Slika 37 Shranjevanje vozlišč, povezav in nastavitvev

Pri serializaciji vozlišč se uporablja enostaven razred, ki pomaga pri pretvorbi podatkov vozlišča v JSON format.

Ko uporabnik uvozi projekt, *NodeLoadingManager* odpre in prebere shranjeno datoteko. Po vrsti prebere vsak zapis in glede na tip zapisa ustvari primeren objekt ali spremeni nastavitve. Ker so v datoteki najprej zapisani podatki o vozliščih, se najprej ustvarijo vozlišča, šele nato pa povezave med njimi. Če bi aplikacija ustvarjala povezave, preden se ustvarijo vsa vozlišča, bi lahko prišlo do napak, saj povezava med neobstoječimi vozlišči ni mogoča.

11. Zaključek

Glavni cilj naloge je bil dosežen, saj aplikacija deluje in sem jo med in po razvijanjem pogosto preizkušal. Omogoča generacijo terena z uporabo različnih funkcionalnosti in njihovo povezovanje v grafu, kar daje uporabniku fleksibilnost pri ustvarjanju različnih oblik terena. Izvedba operacij bi lahko bila hitrejša, vendar bi to zahtevalo izvajanje operacij nad terenom na grafičnem procesorju, kar je zahtevnejše za implementacijo in presega obseg tega projekta.

12. Viri in literatura

- Hans Theobald Beyer, 2015, *Implementation of a method for hydraulic erosion*. <http://www.firespark.de/resources/downloads/implementation%20of%20a%20method%20for%20hydraulic%20erosion.pdf> (5. 3. 2026)
- <https://gamedev.stackexchange.com/questions/116205/terracing-mountain-features> (14. 2. 2026)
- <https://stackoverflow.com/questions/4887530/struggling-with-vertex-and-index-buffers-in-direct3d> (8. 4. 2026)
- Inigo Quilez, 2002, *Domain warping*. <https://iquilezles.org/articles/warp/>
- Victor Karp, 2025, *Cheap Godot terrain shader*. <https://victorkarp.com/godot-terrain-shader/>
- Weisstein, Eric W. "Vertex." iz *MathWorld* - A Wolfram Resource. <https://mathworld.wolfram.com/Vertex.html> (8. 4. 2026)
- https://youtube.com/playlist?list=PLeCKjxofwyfhBJelhPzo96y_nvLTZmJGy (1. 12. 2025)
- <https://docs.godotengine.org/> (1. 12. 2025)
- <https://github.com/Auburn/FastNoiseLite> (2. 12. 2025)
- <https://www.redblobgames.com/maps/terrain-from-noise> (2. 12. 2025)
- <https://github.com/SebLague/Hydraulic-Erosion> (5. 3. 2026)

IZJAVA O AVTORSTVU

Izjavljam, da je strokovno poročilo *Aplikacija za proceduralno generiranje višinskih zemljevidov in digitalnih modelov terena* v celoti moje avtorsko delo, ki sem ga izdelal samostojno s pomočjo navedene literature in pod vodstvom mentorja.

Ljubljana, 16. 4. 2026

Nik Pavlović-Semen